

Documentación Técnica: TAD E-commerce Magenta

Pablo Saez Morales
Oleksandr Baranets
Nicolas Porra Collado

10 de Mayo de 2026

Repositorio: <https://github.com/pablosaez21/TAD-Ecommerce>
Grupo Magenta: Pablo Saez Morales, Oleksandr Baranets, Nicolas Porra Collado

Índice general

1. Resumen ejecutivo	3
2. Objetivos y alcance	4
2.1. Objetivos	4
2.2. Alcance	4
3. Arquitectura y diseño	5
3.1. Visión general	5
3.2. Modelos principales	5
3.2.1. Detalles y extractos de código	5
3.3. Esquema de base de datos	6
4. Rutas y controladores	7
4.1. Rutas principales	7
4.2. Controladores clave	7
4.2.1. CartController: métodos clave	7
5. Proceso de compra (checkout)	9
5.1. Flujo normal	9
5.2. Manejo de errores y rollback	9
6. Decisiones técnicas y de diseño	10
6.1. Autenticación	10
6.2. Carrito en sesión	10
6.3. Transacciones y concurrencia	10
6.4. Internacionalización	10
7. Estructura de ficheros relevante	11
8. Instalación y puesta en marcha	12
8.1. Requisitos	12
8.2. Pasos rápidos	12
9. Configuración de correo	13
10. Pruebas	14
10.1. Pruebas unitarias y funcionales	14
11. Seguridad	15

12.Eliminación de contenido irrelevante	16
13.Decisiones de mejoras futuras	17
14.Apéndice: fragmentos de código relevantes	18
14.1. Ejemplo: Transacción de checkout (extracto)	18
15.Conclusión	19

Capítulo 1

Resumen ejecutivo

TAD E-commerce Magenta es una aplicación de comercio electrónico desarrollada con **Laravel 9** que soporta catálogo de productos, carrito en sesión, proceso de checkout con transacciones ACID, gestión de direcciones, panel administrativo y sistema multiidioma (es/en). Esta documentación recoge diseño, decisiones técnicas, estructura, instrucciones de instalación y uso, pruebas y conclusiones.

Capítulo 2

Objetivos y alcance

2.1. Objetivos

- Implementar un e-commerce funcional con autenticación segura (Fortify).
- Asegurar coherencia de datos en el checkout mediante transacciones y bloqueo de stock.
- Proveer un panel administrativo para gestión de catálogo y pedidos.
- Mantener buenas prácticas de seguridad y rendimiento.

2.2. Alcance

El alcance cubre las funcionalidades implementadas en el repositorio actual: modelos, migraciones, controladores, vistas Blade, rutas, internacionalización y envíos de correo (Mailables). No incluye integraciones de terceros en producción (pasarelas de pago reales) aunque la arquitectura permite su incorporación.

Capítulo 3

Arquitectura y diseño

3.1. Visión general

La aplicación sigue el patrón MVC de Laravel: modelos Eloquent para la lógica de datos, controladores para orquestación, vistas Blade para presentación. Los assets se gestionan con Vite y npm.

3.2. Modelos principales

Resumen de modelos y relaciones (implementadas en `app/Models`):

- **User**: relaciones `hasMany(addresses, orders)`, `belongsToMany(products)` via `favorites`.
- **Product**: relaciones `belongsToMany(categories)`, `hasMany(orderLines)`, campos: `name`, `description`, `price`, `stock`, `active`.
- **Category**: `belongsToMany(products)`.
- **Order**: `belongsTo(user, address)`, `hasMany(orderLines)`, `hasOne(payment)`.
- **OrderLine**: `belongsTo(order, product)`, guarda `quantity`, `unit_price`, `subtotal`. `Address` : `belongsTo(user)` *con campos de dirección y is_default*. `Payment` : `belongsTo(order)` *registra el método y es*

3.2.1. Detalles y extractos de código

Los siguientes extractos provienen de los ficheros reales en `app/Models` y ayudan a entender decisiones concretas.

Producto (`app/Models/Product.php`)

El modelo define los campos rellenables y las relaciones principales:

```
1 protected $fillable = ['name', 'description', 'price', 'image', 'stock', 'active'];
2
3 public function categories(): BelongsToMany { return
    ↪ $this->belongsToMany(Category::class); }
4 public function orderLines(): HasMany { return
    ↪ $this->hasMany(OrderLine::class); }
```

```
5 public function favoredByUsers(): BelongsToMany { return
    ↪ $this->belongsToMany(User::class, 'favorites'); }
```

Notas:

- El campo `price` se castea a decimal con dos decimales para evitar errores de formato.
- La relación `favoredByUsers` usa la tabla pivote `favorites`.

Orden (app/Models/Order.php)

El modelo de orden relaciona líneas, usuario, dirección y pago:

```
1 protected $fillable = ['user_id', 'address_id', 'status', 'total'];
2
3 public function lines(): HasMany { return $this->hasMany(OrderLine::class); }
4 public function payment(): HasOne { return $this->hasOne(Payment::class); }
```

Observaciones:

- El total se almacena con precisión decimal (`decimal:2`).
- Se mantiene copia de precio en `order_lines` para auditoría histórica.

3.3. Esquema de base de datos

Se aplica normalización hasta 3NF; las tablas pivot eliminan redundancia en relaciones N:M (ej. `favorites`, `category_product`, `discount_product`). Las decisiones principales:

- Precios se copian en `order_lines` para auditoría histórica.
- Las claves foráneas usan `onDelete('cascade')` cuando procede.
- Índices en columnas de búsqueda (`created_at`, `status`).

Capítulo 4

Rutas y controladores

4.1. Rutas principales

Rutas definidas en `routes/web.php` (resumen):

- **PÚBLICAS:** GET `/`, GET `/products`, GET `/products/product`, GET `/categories/slug`.
- **AUTENTICADAS:** GET/POST `/cart`, POST `/cart/checkout`, perfil y direcciones, favoritos.
- **ADMIN:** rutas protegidas con middleware `admin` para `/admin` y CRUD de productos y categorías.

4.2. Controladores clave

- **ProductController:** list, show, create, store, edit, update, destroy (admin + público).
- **CartController:** gestionar carrito en sesión, validar stock, checkout (transacción).
- **ProfileController:** gestionar direcciones, idioma, ver pedidos.
- **FavoriteController:** toggle favoritos.
- **DashboardController:** estadísticas administrativas.

4.2.1. CartController: métodos clave

Se muestran extractos reales de `app/Http/Controllers/CartController.php` y una explicación del comportamiento:

Añadir al carrito (add)

El método valida disponibilidad del producto y mantiene el carrito en sesión. Fragmento:

```

1 if (! $product->active || $product->stock <= 0) {
2     return back()->with('error', 'Este producto no est disponible.');
```

```

3 }
4
5 $cart = session('cart', []);
6 $cart[$product->id] = [... 'quantity' => min($quantity, $product->stock) ...];
7 session(['cart' => $cart]);

```

Notas:

- El carrito se guarda como array asociativo en la sesión: clave = id del producto.
- La cantidad se clampa por el stock disponible para evitar solicitudes inválidas.

Checkout (checkout)

El checkout es el núcleo del flujo de negocio: usa validación, transacción y bloqueo pessimistico para stock.

```

1 DB::transaction(function () use ($validated, $user, $address) {
2     $order = Order::create([...]);
3     foreach ($validated['items'] as $item) {
4         $product = Product::whereKey($item['product_id'])
5             ->where('active', true)
6             ->lockForUpdate()
7             ->firstOrFail();
8
9         if ($product->stock < $item['quantity']) {
10             abort(422, "Stock insuficiente para {$product->name}");
11         }
12
13         $order->lines()->create([ ... ]);
14         $product->decrement('stock', $item['quantity']);
15     }
16     $order->payment()->create([...]);
17     return $order->fresh(['lines.product', 'payment', 'address']);
18 });

```

Explicación:

- Se usa `lockForUpdate()` para bloquear la fila del producto durante la transacción.
- Si falta stock, se aborta con código 422, provocando rollback.
- La creación del pago se realiza dentro de la misma transacción para mantener consistencia.

Capítulo 5

Proceso de compra (checkout)

5.1. Flujo normal

1. Usuario autenticado inicia checkout desde carrito en sesión.
2. Sistema valida stock de cada producto con bloqueo (`lockForUpdate`).
3. Se crea la orden, `order_lines` y registro de payment dentro de una `DB::transaction`.
4. Se decrementa stock y se actualiza total.
5. Se envía correo de confirmación (Mailable) y se limpia el carrito.

5.2. Manejo de errores y rollback

Cualquier fallo en la transacción provoca rollback. Casos alternativos: carrito vacío, stock insuficiente, fallo en envío de correo (reintentos asincrónicos / logging).

Capítulo 6

Decisiones técnicas y de diseño

6.1. Autenticación

Se eligió **Laravel Fortify** para autenticación y gestión de credenciales; permite 2FA y acciones personalizadas (`CreateNewUser`, `UpdateUserPassword`, etc.).

6.2. Carrito en sesión

Ventaja: simple, rápida y suficiente para la EPD. Alternativa (persistente en BD) considerada para futuras versiones.

6.3. Transacciones y concurrencia

Uso de transacciones ACID y `lockForUpdate` para evitar sobreventa de stock en escenarios concurrentes.

6.4. Internacionalización

Se utilizan archivos PHP en `resources/lang/es` y `resources/lang/en`. La preferencia de idioma se guarda en el campo `language` del usuario.

Capítulo 7

Estructura de ficheros relevante

- `app/Models/`: modelos Eloquent.
- `app/Http/Controllers/`: controladores.
- `resources/views/`: plantillas Blade (productos, carrito, checkout, perfil, admin).
- `routes/web.php`: rutas web.
- `database/migrations/`: migraciones y seeders.
- `app/Mail/`: clases Mailable (OrderConfirmationMail).

Capítulo 8

Instalación y puesta en marcha

8.1. Requisitos

- PHP 8.1+
- Composer
- Node.js 16+ y npm
- MySQL o PostgreSQL

8.2. Pasos rápidos

```
# clona el repo (local ya presente)
cd /home/miller/Desktop/tad/TAD-Ecommerce
composer install
cp .env.example .env
php artisan key:generate
# configurar .env con credenciales DB y MAIL
php artisan migrate --seed
npm install
npm run dev
php artisan serve
```

over: <http://localhost:8000>

Capítulo 9

Configuración de correo

Se recomienda Mailtrap para desarrollo o configuraciones SMTP en .env ($MAIL_HOST$, $MAIL_PORT$,

Capítulo 10

Pruebas

10.1. Pruebas unitarias y funcionales

Tests incluidos en `tests/Feature` y `tests/Unit` cubren: modelos, checkout, creación de órdenes y validaciones básicas. Ejecutar:

```
php artisan test
```


Capítulo 11

Seguridad

- Contraseñas cifradas (bcrypt).
- Protección CSRF en formularios (Blade @csrf).
- Validación en servidores mediante FormRequest.
- Middleware de autorización (admin) para rutas sensibles.

Capítulo 12

Eliminación de contenido irrelevante

Se han eliminado todas las referencias a la documentación y recursos del proyecto previo ("Sabores de Barrio"). No quedan inclusiones de imágenes externas que pertenecieran a ese proyecto. Si existen ficheros de imagen en el repositorio que no pertenecen a TAD E-commerce y deseas que los borre del disco, indícalo y procederé con cuidado.

Capítulo 13

Decisiones de mejoras futuras

- Integrar pasarela de pago real (Stripe/PayPal).
- Carrito persistente en BD para sesión multi-dispositivo.
- API REST y microservicios para pagos y notificaciones.
- Tests de integración y CI/CD en GitHub Actions.

Capítulo 14

Apéndice: fragmentos de código relevantes

14.1. Ejemplo: Transacción de checkout (extracto)

```
1 DB::transaction(function() use ($cart, $user, $validated) {
2     foreach ($cart as $item) {
3         $product = Product::where('id', $item['id'])
4             ->lockForUpdate()->first();
5         if ($product->stock < $item['quantity']) {
6             throw new Exception('Stock insuficiente');
7         }
8     }
9     $order = Order::create([...]);
10    // crear order lines, decrementar stock, crear payment
11 });
```

Capítulo 15

Conclusión

Esta documentación proporciona una base completa y profesional del proyecto TAD E-commerce Magenta. Si quieres, puedo:

- Añadir diagramas simplificados (tablas) para arquitectura y casos de uso.
- Generar un PDF (compilar) desde aquí — avisando que la compilación puede fallar si faltan paquetes del sistema.
- Eliminar físicamente ficheros de imagen no deseados del repositorio.

Documento preparado por Grupo Magenta — Mayo 2026